

.inputrc: VI experience in the shell

.inputrc: VI experience in the shell, published in deuterium's blog.

SECTION	PUBLISHED	TAGS
deuterium's blog	2024-01-28	4

As a VIM lover, have you ever wondered the possibility of using vim bindings to edit commands in your shell? Or perhaps you're curious about how to manage lengthy commands effortlessly, without getting lost in endless flag combinations? If so, buckle up! You're in for a treat!

In this blog post, we will introduce you to an indispensable yet often overlooked component of the Bash environment - the .inputrc file. This secret sauce allows users to customize readline, the built-in line editor for Bash, unveiling hidden productivity boosters and streamlining command-line workflows.

Say goodbye to monstonsous manual edits and hello to a powerful editing experience inspired by vim!

Toggling modes

VIM has several modes, but we will care about 3 modes here

1. **Normal mode** - The "default" mode of vim (or when you press `Esc`). In this mode, the keyboard inputs control movements, commands, and operators instead of inserting text

2. **Insert mode** - Accessed by pressing certain buttons like `i` in Normal mode. This mode enables to input text, allowing to type characters (as you would expect in any single mode text editor)
3. **Visual mode** - Accessed by pressing `v` in normal mode. This allows you to select blocks of text for operations like copy (yank `y`), delete `d` or changing the content `c`. Similarly, we will use 3 modes in our commandline.
4. By default the command line will be in command (or normal) mode (marked by the prepended `C` to indicate command mode and have a blinking block in my configuration).
5. Insert mode similarly triggered by pressing buttons like `i`, `a` etc in command mode
6. Pressing `v` to edit commands in visual mode? (more about it later)

Herein lies an exciting opportunity! Instead of pressing the `Esc` key each time to alternate between insert and command modes, let's bind a convenient button or sequence of buttons for seamless transitions. I personally recommend using `;;`, a combination that resides comfortably within the home row of your keyboard and is not commonly used in other command sequences.

However, please be aware that when typing `;;` while in insert mode, it's essential to allow a brief pause between the two key presses to avoid triggering any unintended actions

Useful editing motions in vim

There are a couple of motions and commands which you would like to (or you might have unconsciously wished to) leverage in your command line, but they

need to be configured in `.inputrc`

1. **Repeating the last action or command** - By default, pressing `.` to repeats the last change made. You can bind `!!` command of `bash` to re-input the last run command.
2. **Deleting text** - VIM provides several ways to delete text quickly. For example, `dd` to easily delete the entire line and `D` (i.e `shift-d`) to delete the text in front of the cursor.
3. **Additional groupings** - Additionally other text groupings like forward word `w`, backward word `b`, inside something `i` can be paired with delete `d` (e.g `dw`) to quickly delete specific texts groups
4. **Changing stuff** - Even easier, just delete the said stuff and go in insert mode, no need to manually type `i` with same combinations of the text groups
5. **Finding characters**: Vim allows you to find a character forward (`f`) or backwards (`F`) to the cursor. Once you find the character, you can use `;` to go to the next occurrence and `,` to the previous occurrence

NOTE: incremental search with `/` and `?` just like vim is not really supported

My thoroughly commented inputrc

BASH

176 lines / static highlight

```
# TURN ON VIM (E.G. FOR READLINE)
set editing-mode vi
```

SHOW THE VIM MODE IN THE PROMPT (COMMAND OR INSERT)

set show-mode-in-prompt on

SET THE MODE STRING AND CURSOR TO INDICATE THE VIM MODE

FOR THE NUMBER AFTER `\\e[`:

0: blinking block

1: blinking block (default)

2: steady block

3: blinking underline

4: steady underline

5: blinking bar (xterm)

6: steady bar (xterm)

distinguish between command and insert mode

Prepend I to indicate insert mode and have blinking bar

set vi-ins-mode-string I\\1\\e[5 q\\2

prepend C to indicate command mode and have a blinking block

set vi-cmd-mode-string C\\1\\e[1 q\\2

set blink-matching-paren on

*# sets the readline to display possible completions using
different colors*

to indicate filetypes determined from env variable LC_COLORS

set colored-stats on

*# Completions listed immediately instead of bell when completing
word has more than one possible completion*

set show-all-if-ambiguous on

set completion-ignore-case on

set menu-complete-display-prefix on

*# Enables the display of only the first 5 unique characters from
a group of file or directory name suggestions when you utilize
tab-completion.*

If multiple entries share a long prefix, they will be condensed

using ellipses, making it easier to locate and choose the desired item.

Useful in directories containing many similarly named images or documents.

```
set completion-prefix-display-length 5
```

Set the bell-style to be visible only i.e no audio played on command completion

can also be set to none

```
set bell-style visible
```

```
$if mode=vi
```

```
#vi mode settings
```

```
set keymap vi-command
```

go into insert mode, re run last command with !! and press enter

```
".": "i!!\r"
```

```
"|": "A | "
```

delete rest of the line (vi "D" behavior)

```
"D": kill-line
```

change line -> delete then go in insert mode

```
"C": "Da"
```

```
"dw": kill-word
```

```
"dd": kill-whole-line
```

```
"db": backward-kill-word
```

delete and change line

```
"cc": "ddi"
```

change word

```
"cw": "dwi"
```

change backward word

```
"cb": "dbi"
```

vi equivalent of delete all word i.e delete the current word entirely

```
"daw": "lbdW"
```

```
"yaw": "lbyW"
```

change all word, delete and edit the current word

"caw": "lbcW"

delete inner word (word under the cursor without the surrounding whitespaces)

"diw": "lbdw"

yank inner word

"yiw": "lbyw"

change inner word

"ciw": "lbcw"

delete around double quoted string -> delete the text in double quoted strings and the quotes themselves

F search backward for a double quote, then delete till first forward search of double quotes

"da\\": "lF\\df\\"

delete inside double quoted string -> delete the text inside the double quoted strings but not the quotes

"di\\": "lF\\lmtf\\d`t"

change inside double quoted string basically delete inside double quoted string and go in insert mode

"ci\\": "di\\i"

change around double quoted string

"ca\\": "da\\i"

delete around single quoted string

"da'": "lF'df'"

"di'": "lF'lmtf'd`t"

"ci'": "di'i"

"ca'": "da'i"

delete around tilde

"da`": "lF\\`df\\`"

"di`": "lF\\`lmtf\\`d`t"

"ci`": "di`i"

"ca`": "da`i"

delete around parenthesis

"da(": "lF(df)"

"di(": "lF(lmtf)d`t"

"ci(": "di(i"

"ca(": "da(i"

"da)": "lF(df)"

"di)": "lF(lmtf)d`t"

"ci)": "di(i"

"ca)": "da(i"

delete around curly

"da{": "lF{df}"

"di{": "lF{lmtf}d`t"

"ci{": "di{i"

"ca{": "da{i"

"da}": "lF{df}"

"di}": "lF{lmtf}d`t"

"ci}": "di}i"

"ca}": "da}i"

delete around square brackets

"da[": "lF[df]"

"di[": "lF[lmtf]d`t"

"ci[": "di[i"

"ca[": "da[i"

"da]": "lF[df]"

"di]": "lF[lmtf]d`t"

"ci]": "di]i"

"ca]": "da]i"

delete around angled brackets

"da<": "lF<df>"

"di<": "lF<lmtf>d`t"

"ci<": "di<i"

"ca<": "da<i"

"da>": "lF<df>"

```
"di>": "lF<lmtf>d`t"
```

```
"ci>": "di>i"
```

```
"ca>": "da>i"
```

```
# delete around forward slash
```

```
"da/": "lF/df/"
```

```
"di/": "lF/lmtf/d`t"
```

```
"ci/": "di/i"
```

```
"ca/": "da/i"
```

```
# delete around colon
```

```
"da:": "lF:df:"
```

```
"di:": "lF:lmtf:d`t"
```

```
"ci:": "di:i"
```

```
"ca:": "da:i"
```

```
"gg": beginning-of-history
```

```
"G" : end-of-history
```

```
# backward history search on up arrow
```

```
"\e\e[A": history-search-backward
```

```
# forward history search on down arrow
```

```
"\e\e[B": history-search-forward
```

```
# settings to be changed in insert mode
```

```
set keymap vi-insert
```

```
";;": vi-movement-mode
```

```
# using ;; to easily move to command mode from insert mode  
instead of going to press Escape key each time
```

```
# ;; as I find it relatively easily on the home row and it  
doesn't interfere with typing in insert mode
```

```
# as commands to come by with ;; are rare?
```

```
TAB: menu-complete
```

```
# shift tab to menu complete backward
```

```
"\e[Z": menu-complete-backward
```



```
# end vi mode settings  
$endif
```

Check the latest version [in my dotfiles \(https://github.com/deut-erium/.dotfiles/blob/main/.inputrc\)](https://github.com/deut-erium/.dotfiles/blob/main/.inputrc) in an event I forget to update my findings here

How to source `.inputrc`

You can instruct `readline` to re-read the `.inputrc` file

BASH

1 line / static highlight

```
bind -f ~/.inputrc
```

Alternatively you can change the keymap of your preference to `re-read-init-file`

BASH

3 lines / static highlight

```
# in .inputrc  
# press control-x control-r (which should be the default)  
"\C-x\C-r": re-read-init-file
```

Editing long commands

Are you having trouble editing lengthy bash commands, such as those with numerous flags, and finding it tedious to search for and navigate to the specific

section that needs modification?

Fear not! Pressing `v` in visual mode (you guessed it right, cause its vim) allows you to input the current command into a preferred text editor (as specified in your `$EDITOR` environment variable, shame if its not set to vim in your `.bashrc`).

With this feature, you can efficiently make the necessary changes leveraging the full powers of your favorite text editor. Upon saving the edited text, the updated command will be executed. No more wasted time manually updating flags while risking errors; breathe new life into repetitive, mundane workflows.

Join the ranks of true CLI wizards by embracing this underutilized technique, today. Witness the seemingly insurmountable challenge of managing convoluted commands

Caveats

1. Keep in mind that quitting without saving (from the visual mode) will still execute the original command.

TIP: Adding a “#” at the beginning of a command can help prevent its accidental execution!

2. Multiline commands, separated by a backslash (“\”), may appear differently in searchable command history due to replacement with spaces. For example, entering `echo hello \n world` becomes `echo hello world` in the history log.

PRO TIP: Make it a habit to review your entered command before executing to ensure accuracy and consistency in your workflow!

3. Separate entries are created for multiple commands in history search, meaning each command is treated independently, regardless of whether they were entered together or not.

References

- [Bash command line editing manual \(https://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html#Command-Line-Editing\)](https://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html#Command-Line-Editing)
- [Usenix bash notes \(https://www.usenix.org.uk/content/bash.html#input\)](https://www.usenix.org.uk/content/bash.html#input)
- [Superuser, how do i reload inputrc \(https://superuser.com/questions/241187/how-do-i-reload-inputrc\)](https://superuser.com/questions/241187/how-do-i-reload-inputrc)
- [Inputrc for humans \(https://www.topbug.net/blog/2017/07/31/inputrc-for-humans/\)](https://www.topbug.net/blog/2017/07/31/inputrc-for-humans/)
- [The very basics of vim \(https://www.danielmsullivan.com/pages/tutorial_vim.html\)](https://www.danielmsullivan.com/pages/tutorial_vim.html)

[Suggest a correction to this article.](#)